

Table des matières

Corrigé 01 — Le métier, les sources et le premier regard	1
Questions courtes	1
Questions pratiques	4
Diagnostics	7
Questions à développer	8
Mises en situation	9

Corrigé 01 — Le métier, les sources et le premier regard

Corrigé des exercices de `Exercices_01.md`. La numérotation est la même. Les chiffres sont ceux de la génération par défaut de Boréal ; les temps, ceux du poste de l'auteur.

Questions courtes

1. La donnée arrive (tout ce que la source a produit est là), elle est rangée (au bon endroit, dans un format lisible), elle est à l'heure (disponible quand on en a besoin, et l'on sait de quand elle date), elle est bonne (des contrôles le prouvent). La propriété qui tient le tout : un traitement se rejoue sans rien casser, ni doubler ni perdre. C'est l'idempotence, qui gouverne la formation à partir du Cours 03.

Théorie : `Theorie_01.md`, section 1.

2. Trois parmi cinq. L'ingénieur analytique attend une couche RAW complète et à jour. L'analyste et les opérations attendent des tableaux de bord justes, certains en direct. Le scientifique de données attend des séries longues et propres, qu'il peut rejouer pour réentraîner un modèle. L'ingénieur ML attend les mêmes données à l'entraînement et en production. L'ingénieur IA attend une documentation complète, dont il connaît la version.

Théorie : `Theorie_01.md`, section 1.

3. Source (l'ERP, le SCADA et son historien, la GMAO, le portail des garanties, les véhicules connectés, l'API météo) → ingestion (scripts Python, Airflow, MQTT, Kafka) → lac (Parquet, Iceberg) → entrepôt (DuckDB : la couche RAW et les séries de l'usine) → restitution (Grafana en direct, les rapports chez BI).

Théorie : `Theorie_01.md`, section 2.

4. Le lac est un stockage de fichiers : tout format y entre, pour presque rien, et la structure se décide à la lecture. L'entrepôt est une base de tables : un schéma défini d'avance, en échange du SQL, des types et des droits.

Théorie : Theorie_01.md, section 2.

5.

- (a) Froid : un chiffre du mois passé, qui doit être complet et exact, et qu'un lot calcule très bien. (b) Chaud : la décision (arrêter le compresseur, dépêcher un technicien) se prend dans la minute. (c) Froid : une prévision se calcule sur l'historique, une fois par jour suffit largement.
- (b) Chaud, avec une nuance qui compte : ce besoin est déjà servi par le SCADA en salle de contrôle. Le pipeline de données n'a pas à refaire ce que le système de conduite fait déjà.

Théorie : Theorie_01.md, section 3.

- 6. Parce que ses services tournent jour et nuit, qu'il y ait des mesures ou non, et qu'ils doivent être dimensionnés pour la pointe. Un lot s'allume, calcule et s'éteint. S'ajoute un coût d'ingénierie : le continu oblige à traiter les données en retard, les doublons de livraison et les pannes en cours de route, que le lot règle simplement en rejouant.

Théorie : Theorie_01.md, section 3.

- 7. Beaucoup de demandes présentées comme urgentes tolèrent en réalité un délai d'une heure, voire d'un jour. La question à poser : quelle décision sera prise dans la minute qui suit la mesure, et par qui ? S'il n'y en a pas, un lot fait l'affaire.

Théorie : Theorie_01.md, section 3.

- 8. À la couche RAW : le brut tel que la source l'a livré, complet, daté, avec la trace de son origine. L'ingénieur analytique prend le relais au STAGING. L'exception : les séries de terrain (l'historien, le compteur électrique). Les nettoyer (valeurs de départ recopiées, heures qui se répètent au changement d'heure, codes de qualité) demande de connaître la source, et c'est l'ingénieur de données qui la connaît. Il livre donc aussi ces séries nettoyées, dans le schéma argent.

Théorie : Theorie_01.md, section 4.

- 9. La règle du brut intact : on ne transforme pas à l'ingestion, la couche RAW reste le miroir de la source. Ses conversions ne seraient ni versionnées, ni testées, ni visibles dans le lignage, et le jour où l'une se tromperait (une date vide 00000000 convertie en une date absurde, par exemple), le brut d'origine n'existerait plus. Décoder les codes et typer les dates, c'est le travail du STAGING de l'ingénieur analytique, dans du code relu.

Théorie : Theorie_01.md, section 4.

- 10. L'ERP (un ERP de type SAP), l'usine (le SCADA et son historien, le registre des actifs, la GMAO, la gestion de l'énergie), les garanties (le portail des garanties et de l'après-vente), la télémétrie (le module télématique des véhicules connectés), la documentation (la documentation technique et commerciale), la météo (Environnement et Changement climatique Canada, par API REST).

Théorie : Theorie_01.md, section 5.

11. `UTL.CMP2.TEMP_REFOULEMENT` : la température de l'air au refoulement du compresseur d'air 2, dans la zone des utilités. `UTL-CMP2` : le compresseur d'air 2 lui-même, l'équipement. La différence d'écriture sépare deux choses : un identifiant d'actif porte un tiret (la zone, l'équipement), une étiquette de l'historien des points (la zone, l'équipement, la grandeur). Le référentiel (`etiquettes.csv`) donne la description et l'unité de chaque étiquette, et la relie à son actif.

Théorie : Theorie_01.md, section 5.

12. Ce sont les codes de qualité hérités d'OPC, le standard des équipements industriels : 192 pour une valeur bonne, 64 pour une valeur douteuse, 0 pour une valeur mauvaise, qui arrive sans valeur.

Théorie : Theorie_01.md, section 5.

13. La météo. Première raison : une API réelle a des comportements qu'une simulation imite mal (pagination, limites de débit, pannes passagères), et le Cours 04 veut justement apprendre à extraire de façon fiable face à eux. Seconde raison : la vraie météo pilote le fictif. Les ventes de motoneiges suivent la neige au sol, la demande électrique de l'usine suit la température de Bagotville. C'est ce lien qui rendra honnête la prévision de la formation Data Science.

Théorie : Theorie_01.md, section 5.

14. Deux parmi : il n'y a pas de serveur à installer, démarrer ni sécuriser, il se charge comme une bibliothèque Python ; il lit CSV, JSON et Parquet sur place, sans étape de chargement ; il tient sur un portable ordinaire, même sur des volumes qui étoufferaient un tableur.

Théorie : Theorie_01.md, section 6.

15. `generer_boreal.py` vit dans le dépôt des formations, sous `Data_Domain/_univers/boreal/` : c'est un outil partagé par les sept formations du parcours, en un seul exemplaire. `sources/` ne vit dans aucun dépôt : il se trouve dans `Projet_Boreal`, que le dépôt des formations ignore, et à côté de `01_de`, pas dedans. Le code de ce cours vit dans votre dépôt `01_de`.

Théorie : Theorie_01.md, section 4.

16. Le générateur supprime `sources/` et le refait de zéro, avec le même tirage : on obtient exactement les mêmes fichiers. La preuve tient en une comparaison : l'empreinte globale qu'il affiche (`37521016091c44da...`) est la même d'une génération à l'autre, et `empreintes.sha256` est identique. Relancer ne double rien et ne perd rien. C'est l'idempotence, que le Cours 03 exigera de chaque ingestion.

Théorie : Theorie_01.md, section 5.

17. 275 436 lignes et 58 étiquettes. Parmi ces lignes, chaque fichier commence par la valeur de départ de chaque étiquette : des lignes recopiées de la veille, qui ne sont pas de nouvelles mesures.

Théorie : Theorie_01.md, section 5.

18. Le fuseau horaire : rien ne dit que l'heure est locale, ni de combien elle s'écarte de l'UTC. Le problème concret arrive le 2 novembre 2025, au retour à l'heure normale : l'heure entre 1 h et 2 h se produit deux fois. Deux points différents d'une même étiquette peuvent donc porter exactement le même horodatage, et seul l'ordre des lignes dans le fichier dit lequel est venu en premier. Le Cours 08 règle ce cas.

Théorie : Theorie_01.md, section 5.

19. Parce qu'on cesse de vérifier. Un outil qui ne devine jamais oblige à déclarer les types, et l'on réfléchit à chaque colonne. Un outil qui devine juste neuf fois sur dix fait croire que tout va bien, et la dixième colonne, fausse, passe en silence dans les calculs.

Théorie : Theorie_01.md, section 7.

20. DuckDB a deviné que ERDAT était un entier : 20251231 plus un fait 20251232, un nombre, pas une date. Le vrai lendemain demande de lire la colonne en texte, puis de la convertir avec son format : `STRPTIME(ERDAT, '%Y%m%d')::DATE + 1` rend 2026-01-01.

Théorie : Theorie_01.md, section 7.

21. La détection regarde les valeurs. Toutes celles de ERDAT ressemblent à des entiers (20240424) : entier. AEDAT vaut souvent 00000000, une date vide ; ses zéros de tête feraient perdre de l'information à un entier, et DuckDB garde le texte. La même sorte de colonne reçoit deux types selon ses valeurs, ce que la documentation SAP ne laisse jamais deviner.

Théorie : Theorie_01.md, section 7.

22. On perd le décalage -05:00, donc le fuseau dans lequel l'heure était exprimée. L'instant reste juste, mais on ne sait plus quelle heure affichait l'horloge locale. Pour `recu_le`, le décalage est toujours celui de l'Est, là où l'abonné reçoit : la perte se répare. Pour `debut` et `fin`, c'est l'heure locale du véhicule : le décalage dit dans quel fuseau il roulait, et le bronze en compte huit, de -08:00 en Colombie-Britannique à -02:30 à Terre-Neuve ; rien d'autre dans le message ne le dit.

Théorie : Theorie_01.md, section 7.

Questions pratiques

P1.

```
SELECT Quality, COUNT(*) AS points
FROM 'sources/usine/historien/*.csv'
GROUP BY Quality
ORDER BY Quality DESC;
```

Résultat : 192 → 275 080 lignes, 64 → 35, 0 → 321. Trier par `Quality DESC` range du meilleur au pire parce que les codes OPC croissent avec la qualité. C'est une coïncidence commode, pas une règle générale : ailleurs, on trierait avec une table de correspondance. Ce décompte porte sur les lignes, valeurs de départ comprises ; le Cours 08 apprend à les écarter.

Théorie : Theorie_01.md, section 5.

P2.

```
SELECT TagName, COUNT(*) AS points
FROM 'sources/usine/historien/*.csv'
GROUP BY TagName
ORDER BY points DESC
LIMIT 5;
```

En tête : ESS.BANC.TEMP_REFROIDISSEMENT (27 661), UTL.CHF.PUISSANCE (24 968), LMN.VIS.COUPLE (9 277), ESS.BANC.REGIME (7 908), PEI.CAB.HUMIDITE (7 877).

```
SELECT TagName, COUNT(*) AS points
FROM 'sources/usine/historien/*.csv'
GROUP BY TagName
ORDER BY points, TagName
LIMIT 1;
```

En queue : UTL.CMP3.MARCHE, 210 points. Le second critère de tri (TagName) rend la réponse déterministe en cas d'égalité ; ici, il n'y en a pas, mais une requête qui dépend d'un ordre ne devrait jamais laisser le hasard trancher. L'écart s'explique par la bande morte : une température ou une puissance bouge sans cesse et franchit souvent son seuil, alors qu'un état de marche ne s'écrit qu'à son changement, et le compresseur d'appoint démarre rarement.

Théorie : Theorie_01.md, section 5.

P3. 92 jours × 24 heures × 30 lectures par heure = 66 240 lectures par étiquette. ESS.BANC.TEMP_REFROIDISSEMENT en garde 27 661, soit environ 42 % ; UTL.CMP3.MARCHE en garde 210, moins de 0,4 %. Trois conclusions. L'historien compresse, et chaque étiquette a son propre rythme. Compter des points ne mesure donc pas une durée : 27 661 points ne veulent pas dire 27 661 fois deux minutes. Et toute agrégation dans le temps (une moyenne horaire, par exemple) doit tenir compte de ces intervalles irréguliers, sujet du Cours 08.

Théorie : Theorie_01.md, section 5.

P4.

```
# Secrets : mots de passe et jetons. Le seul fichier qui PROTÈGE quelque chose ;
# le mot de passe de la base de l'ERP y vivra dès le Cours 04
.env

# Fichiers compilés par Python : régénérés à chaque exécution, propres à chaque machine
__pycache__/
```

sources/ n'y figure pas parce qu'il n'est pas dans le dépôt : il vit dans Projet_Boreal, à côté de 01_de. Garder les données hors du dépôt de code par construction vaut mieux qu'une exclusion : une ligne de .gitignore s'efface par erreur, un dossier rangé ailleurs ne se versionne pas. Le piège à éviter reste de

confondre les deux familles d'entrées : l'une exclut du rejouable, l'autre protège un secret. L'erreur fréquente est d'exclure soigneusement les fichiers inutiles et de committer un mot de passe.

Théorie : Theorie_01.md, section 4.

P5.

```
SELECT CAST(Timestamp AS DATE) AS jour, MAX(Value) AS vibration_max
FROM 'sources/usine/historien/*.csv'
WHERE TagName = 'UTL.CMP2.VIBRATION'
AND Quality = 192
GROUP BY jour
ORDER BY jour;
```

Jusqu'au 17 octobre, les jours de production, la vibration maximale reste sous 3,0 mm/s ; les fins de semaine, elle tombe sous 0,1 : le compresseur est arrêté. À partir du lundi 20 octobre, elle grimpe chaque jour : 3,66 le 20, 4,19 le 21, 4,76 le 22, 5,26 le 23, 5,92 le 24, puis 8,48 le 27 et 9,23 le 28. Le 29, elle retombe à 0,04 ; le 30, elle revient à 2,53, puis autour de 2,9.

Dans usine/gmao/bons_travail.csv, deux bons sur UTL-CMP2 racontent la même histoire. Le 22 octobre à 9 h 12, BT-2025-00739, correctif de priorité 2 : « Bruit anormal signalé par l'opérateur au compresseur 2. Inspection et écoute, rien constaté. » Le 28 octobre à 10 h 50, BT-2025-00742, correctif de priorité 1 : le compresseur a déclenché sur vibration élevée, la soupape d'admission et le roulement du moteur sont remplacés, et le bon se ferme le 30 octobre à 13 h 36.

Les mesures annonçaient la panne une semaine d'avance, et l'inspection du 22 ne l'a pas vue. C'est exactement le cas d'usage de la maintenance prédictive, que la formation Data Science traitera. Ce qui revient à l'ingénieur de données : que la série de vibration et les bons de travail arrivent complets, bien datés, et fiables par la même clé d'actif. Le filtre `Quality = 192` n'est pas décoratif : sans lui, une valeur douteuse suffirait à fausser un maximum. Et la ligne du 30 septembre, en tête du résultat, est une valeur de départ : elle ne dit rien du 30 septembre.

Théorie : Theorie_01.md, section 5.

P6.

```
SELECT COUNT(*) AS lignes_du_bis
FROM 'sources/erp/extraits/2023-03_bis/VBAK.csv';
```

609 lignes, toutes présentes aussi dans 2023-03. L'écart total est de $26\,538 - 25\,483 = 1\,055$ lignes : 609 viennent de l'extrait livré deux fois, et les 446 autres de commandes modifiées après le mois de leur création. L'ERP extrait chaque mois les commandes créées ou modifiées dans le mois, dans leur version de fin de mois : une commande modifiée le mois suivant revient dans l'extrait suivant, avec sa nouvelle version. Les deux écarts ne se traitent pas pareil. Le premier est une erreur de livraison, que le Cours 03 empêche de doubler et que le Cours 11 signale. Le second est normal : c'est l'historique des versions d'une commande, que la RAW garde et que l'ingénieur analytique réduira à la dernière version.

Théorie : Theorie_01.md, section 5.

P7.

```
SELECT COUNT(*) AS en_tetes,
      MIN(STRPTIME(ERDAT, '%Y%m%d'))::DATE AS premiere_creation,
      MAX(STRPTIME(ERDAT, '%Y%m%d'))::DATE AS derniere_creation
FROM read_csv('sources/erp/extraits/2024-05/VBAK.csv', all_varchar = true);
```

Résultat : 647 en-têtes, créés du 2024-04-24 au 2024-05-31. L'extrait de mai contient les commandes créées ou modifiées en mai : une commande créée fin avril et modifiée en mai y revient, dans sa version de fin de mai. C'est la note de l'équipe de l'ERP, erp/LISEZMOI.md, qui le dit.

Théorie : Theorie_01.md, section 7.

P8.

```
SELECT MIN(DATEDIFF('day', STRPTIME(ERDAT, '%Y%m%d'), STRPTIME(VDATU, '%Y%m%d'))) AS delai_min,
      MAX(DATEDIFF('day', STRPTIME(ERDAT, '%Y%m%d'), STRPTIME(VDATU, '%Y%m%d'))) AS delai_max
FROM read_csv('sources/erp/extraits/*/VBAK.csv', all_varchar = true);
```

Résultat : de 3 à 105 jours. Lues en texte et converties avec leur format, les deux dates donnent de vrais jours, qu'un changement de mois ou d'année ne dérange pas.

Théorie : Theorie_01.md, section 7.

Diagnostics

E1. Le besoin a été confondu avec l'outil. Un rapport mensuel n'appelle aucune décision dans la minute : il a besoin d'être complet et exact, et de pouvoir être refait si on découvre une erreur. C'est précisément ce que donne le chemin froid, et ce que le chaud donne mal. Trois services allumés en permanence coûtent chaque mois ce qu'un lot coûterait en un an, et chacun est une pièce de plus qui peut tomber en panne. Le correctif : un lot mensuel, ou de nuit, qui calcule l'énergie à partir des relevés du compteur. On ne garde le flux que s'il sert aussi une vraie décision immédiate, par exemple une alerte de pointe de puissance.

Théorie : Theorie_01.md, section 3.

E2. Il manque l'idempotence, et le cas est vicieux parce qu'il ne plante pas. L'extrait de mars 2023 a été livré deux fois, sous deux noms de dossier (2023-03 et 2023-03_bis) ; le pipeline charge tout ce qu'il trouve, et ses 609 commandes entrent deux fois. Le correctif consiste à rendre l'ingestion rejouable et méfiante : un registre des extraits déjà chargés, qui refuse de recharger un dossier, et un contrôle qui compare le contenu d'un nouvel extrait à ceux déjà reçus, parce qu'un nom différent ne prouve pas un contenu différent. Rejouer le chargement de mars doit laisser la couche RAW exactement dans le même état. Le Cours 03 construit le registre, le Cours 11 le contrôle.

Théorie : Theorie_01.md, section 2.

E3. On a versionné des données brutes. Git garde tout ce qu'on lui confie, pour toujours, et supprimer le dossier ensuite ne rend pas la place : l'historique le porte encore, et chaque clone le transporte en entier. Le mécanisme mérite d'être dit juste, parce qu'il ne fonctionne pas comme on l'imagine. Git ne range pas des copies : il range chaque fichier sous l'empreinte de son contenu, puis il compresse. Les 138 Mo de sources par défaut pèsent une vingtaine de mébiotets une fois commités, et une génération identique n'ajoute rien du tout, puisque les empreintes ne changent pas. Ce qui gonfle, c'est une suite de générations qui

différent : chacune ajoute ses fichiers changés, pour toujours. Une « sauvegarde » de données régénérables n'a d'ailleurs aucun sens : la vraie sauvegarde, c'est le générateur et sa graine fixe, que l'empreinte globale permet de vérifier. Le correctif, dans l'ordre : retirer le dossier du suivi (`git rm -r --cached sources`), le supprimer du dépôt et travailler sur `Projet_Boreal/sources`, `commit`. Si le dépôt a déjà été poussé et que son poids pose problème, il faut réécrire l'historique pour en purger le dossier, ou recréer le dépôt proprement. Pour de vraies données, la sauvegarde relève du stockage (le lac et sa politique de conservation), jamais de Git.

Théorie : Theorie_01.md, section 4.

E4. Les dates ont été devinées en entiers, et la soustraction a porté sur des nombres. Entre le 2022-11-22 et le 2023-01-06, il y a 45 jours ; entre 20221122 et 20230106, il y a 8 984 unités. Tout délai qui franchit un changement de mois gonfle d'une soixantaine au moins, un changement d'année de près de 9 000 : d'où les 13 109 lignes au-delà de 60. Le calcul ne plante jamais, et les délais d'un même mois restent justes, ce qui rend l'erreur crédible. Le correctif : lire les extraits en texte, convertir ERDAT et VDATU avec `STRPTIME` (... , '%Y%m%d'), puis compter les jours avec `DATEDIFF` (exercice P8). Et une règle pour la suite : une colonne de dates qui arrive en entier est un signal d'alarme.

Théorie : Theorie_01.md, section 7.

Questions à développer

A. L'ingénieur de données tient l'amont de la chaîne : il fait arriver la donnée depuis des systèmes qu'il ne contrôle pas, la range dans des formats ouverts, fait tourner les traitements à l'heure et prouve qu'elle est bonne. Il livre une couche RAW brute et complète, et les séries de terrain qu'il est le seul à savoir nettoyer. L'ingénieur analytique prend la suite : il transforme ce brut en tables métier, nommées en clair, modélisées et testées. Le premier garantit que la matière est là et fiable, le second en fait un produit lisible. La propriété de rejouabilité tient les quatre promesses ensemble parce qu'aucune n'est tenable sans elle. Une donnée qui n'est pas arrivée se rattrape en rejouant l'ingestion ; si rejouer double les lignes, on troque une perte contre un doublon, et la donnée n'est plus bonne. Une correction de contrôle s'applique en rejouant ; si rejouer n'est pas sûr, personne n'ose corriger, et la donnée reste fausse. Un traitement rejouable transforme chaque incident en simple relance.

Théorie : Theorie_01.md, section 1.

B. Le chemin chaud prendrait chaque mesure de vibration au moment où elle est produite, la ferait circuler par un courtier de messages (MQTT, puis Kafka), la comparerait à un seuil et alerterait l'usine en quelques secondes si un compresseur s'emballe. Le chemin froid ramasserait chaque nuit l'export de l'historien, le rangerait dans le lac, et bâtirait l'historique complet des vibrations, sur des mois ou des années : la matière pour comprendre les pannes passées, entraîner un modèle de maintenance prédictive, justifier le remplacement d'un roulement. On garde les deux parce qu'ils ne répondent pas à la même question : l'un sert la réaction, l'autre la compréhension. Le chemin froid fait référence : il est complet, il se corrige et se rejoue. Si le chemin chaud perd des mesures pendant une coupure de communication, l'historien, lui, a gardé ce qu'il a reçu ou marqué le trou d'une qualité mauvaise, et le lot du lendemain remet les pendules à l'heure.

Théorie : Theorie_01.md, section 3.

C. Nettoyer à l'ingestion, c'est décider à la place de tous les consommateurs, sans code relu et sans retour possible. Une ligne de qualité 0 sans valeur n'est pas un déchet : elle enregistre un fait, la perte de communication avec une zone de l'usine à un instant précis. Une analyse de fiabilité du réseau a besoin de ces trous, et une moyenne horaire doit savoir qu'elle repose sur une heure incomplète. Une valeur de qualité douteuse est peut-être fausse, mais la règle qui en

décide (au-delà de quel seuil, pour quelle grandeur) doit être écrite, versionnée et testée : c'est le travail des contrôles du Cours 11. Le mois livré deux fois montre le même principe de l'autre côté : le retirer en silence effacerait la trace d'une erreur de livraison que l'équipe de l'ERP doit connaître, alors qu'un contrôle qui la signale la fait corriger à la source. Enfin, aucune règle d'ingestion ne suffit contre un capteur figé : sa valeur est plausible et sa qualité est bonne, seul l'examen de la série dans le temps le révèle. On charge donc tout, tel quel, et l'on traite plus loin, là où le traitement se voit et se corrige.

Théorie : Theorie_01.md, sections 2 et 4.

Mises en situation

M1. Transport collectif des Deux-Rives. Trois demandes, trois réponses différentes.

Demande	Chemin	Sources	Ce que l'ingénieur de données livre
Retard aux arrêts	chaud	positions GPS, horaires planifiés	le flux des positions, reçu en continu et fiable, joint aux horaires pour produire les retards en quelques secondes
Achalandage par ligne et par heure	froid	validations, horaires	les validations de chaque nuit, ingérées sans perte ni doublon, rangées dans le lac et livrées en RAW
Prévision de la rentrée	froid	validations sur plusieurs années, horaires, calendrier scolaire	des séries longues et rejouables, que le scientifique de données consommera

Le retard aux arrêts est le seul vrai besoin chaud : l'information ne vaut que dans les minutes qui suivent. L'achalandage n'appelle aucune décision immédiate, et ses données n'arrivent de toute façon qu'une fois par nuit. La prévision est un travail de Data Science : l'ingénieur de données ne construit pas le modèle, il garantit la matière. Son travail s'arrête à la couche RAW : positions brutes, validations brutes, horaires tels que publiés. Le calcul de l'achalandage par ligne revient à l'ingénieur analytique, le modèle au scientifique.

Serait acceptable aussi : servir l'achalandage à la journée plutôt qu'à la nuit si la planification le demande, tant que le chemin reste un lot. Serait indéfendable : bâtir l'achalandage sur le flux GPS, qui dit où sont les autobus, pas combien de personnes y montent.

Théorie : Theorie_01.md, sections 2 et 3.

M2. Coopérative électrique de la Rivière-aux-Saules. D'abord ce qui est impossible : la consommation des clients ne peut pas être en temps réel, parce qu'elle n'arrive qu'une fois par nuit. Aucun pipeline, si rapide soit-il, n'affiche une donnée qui n'est pas encore partie du compteur. Un tableau de bord de la consommation par client sera donc à J+1, alimenté par un lot de nuit, et c'est très bien pour la facturation, les profils de consommation et la planification.

Ce qui est possible : la charge des douze postes, que le SCADA mesure en continu. Si le directeur veut suivre la pointe du réseau, par exemple lors des grands froids d'hiver, un chemin chaud sur les mesures des postes a du sens. La proposition : un tableau de bord à deux vitesses, la charge des postes en direct et la consommation des clients au lendemain, avec une étiquette claire sur la fraîcheur de chaque chiffre.

Ce qui ferait changer d'avis : un programme de gestion de la pointe qui exigerait d'agir chez les clients dans l'heure, ou un système de collecte capable de transmettre les lectures en continu. Avant de bâtir quoi que ce soit, la question de ce cours : quelle décision prendra-t-il dans la minute ?

Théorie : Theorie_01.md, section 3.

M3. (avancé) L'usine de traitement. Il faut refuser, pour trois raisons. La sécurité d'abord : le réseau de contrôle d'une usine d'eau potable est isolé du réseau de bureau précisément pour qu'un incident informatique ne puisse pas atteindre les filtres et les pompes. Ouvrir une connexion directe vers les automates, c'est percer cette frontière, et le risque n'est pas une fuite de données, c'est une atteinte au traitement de l'eau. Ensuite la charge : les automates sont dimensionnés pour conduire le procédé, pas pour répondre aux requêtes d'un pipeline, et un client de plus peut perturber leur cycle. Enfin la qualité : l'historien fait un travail que le pipeline devrait refaire, horodatage, codes de qualité, tampon pendant les coupures.

Le vrai besoin est la fraîcheur, et il se sert sans percer la frontière : lire l'historien plus souvent qu'une fois par jour, par son interface de requête plutôt que par ses exports ; ou obtenir une copie de l'historien, ou un courtier de messages, dans une zone tampon prévue entre les deux réseaux, que le pipeline interroge depuis le côté bureau. Le Cours 08 revient sur cette frontière, et le Cours 09 sur le courtier. Serait acceptable aussi : garder l'export quotidien si, à l'examen, aucune décision ne justifie mieux. Serait indéfendable : toute solution qui ajoute un accès entrant au réseau de contrôle.

Théorie : Theorie_01.md, sections 2 et 3.